



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

AIRPORT GATE ASSIGNMENT USING GREEDY ALGORITHM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1221-COMPUTER ARCHITECTURE

to the award of the degree of

BACHELOR OF ENGINEERING

IN

DATA SCIENCE

Submitted by

K Harshika (192473020)

Under the Supervision of

DR. P.Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AIRPORT GATE ASSIGNMENT USING GREEDY ALGORITHM” has been carried out by K Harshika (192473020) under the supervision of DR. . P.Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech IT and B.E CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F.Mary Harin Fernandez
Professor Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. P.Solainayagi
Professor Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105

Submitted for the Capstone Project work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, K Harshika (192473020) SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled AIRPORT GATE ASSIGNMENT USING GREEDY ALGORITHM is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, algorithms, test data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names
K Harshika (192473020)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-

602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contributio n (%)
K Harshika (192473020)	System planning and design	Designed scheduling architecture and flight- gate flow	Verified basic gate assignment cases	Chapter 1 & Chapter 3	%

Total Contribution: 100%

ABSTRACT

An Airport Gate Assignment System defines the allocation of arriving and departing flights to available airport gates while considering flight schedules, gate availability, and operational constraints. Efficient gate assignment is essential because airports have limited gates and overlapping flight schedules can cause conflicts, delays, and inefficient utilization. This project applies a greedy algorithm to provide a simple and practical method for assigning flights to suitable gates.

The proposed Airport Gate Assignment System is a software-based scheduling application designed to demonstrate how a greedy algorithm assigns flights to airport gates. The system represents flights using arrival and departure times and maintains the availability of each gate. Flights are processed in schedule order, and the earliest available suitable gate is selected so that conflicts are minimized and available gates are utilized efficiently.

The system is developed using a high-level programming language with a scheduling module that reads flight details, sorts them by arrival time, checks gate availability, and assigns each flight to an available gate using greedy selection. The implementation includes flight input, gate-status tracking, conflict checking, assignment, and result display. The system maintains the assignment state after every flight so that the scheduling process can be understood step by step.

Testing is performed using different flight schedules and numbers of gates. The results demonstrate that the system correctly assigns flights without overlapping use of the same gate and identifies cases in which all gates are occupied. The project provides a practical way to visualize greedy scheduling decisions and understand how local choices can produce an efficient gate-assignment schedule.

The developed system can be used as an educational tool for students to understand greedy algorithms, interval scheduling, resource allocation, and algorithmic decision-making. It also provides a foundation for future extensions such as priority-based gates, aircraft type constraints, terminal-specific gates, delay handling, and optimization using more advanced scheduling techniques.

Keywords

Airport Gate Assignment, Greedy Algorithm, Flight Scheduling, Gate Allocation, Resource Optimization, Algorithm Design

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Algorithm Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure1.1	Basic Architecture of the Airport Gate Assignment System	6
Figure1.2	Greedy Gate Assignment Process	9
Figure 3.1	Overall System Architecture of the Gate Assignment System	12
Figure 3.2	System Flowchart	13
Figure 3.3	Greedy Assignment Flow	15
Figure 3.4	Flight and Gate Status Organization	16
Figure 4.1	System Development Workflow	18
Figure 4.2	Gate Assignment Program Interface	19
Figure 4.3	Gate Assignment Output	20
Figure 4.4	Gate Availability State After Assignment	22
Figure 4.5	Flight Schedule State After Assignment	25
Figure 4.6	Sample Flight Schedule Execution	26
Figure 4.7	Final Gate Assignment Output	27

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1.1	Summary of Project Requirements	6
Table 2.1	Comparison of Existing Gate Assignment Approaches	7
Table 3.1	Stakeholder and User Requirements	8
Table 3.2	Functional and Non-Functional Requirements	11
Table 3.3	Applicable Constraints and Standards	12
Table 3.4	Design Specifications of the Gate Assignment System	13
Table 3.5	Alternative Solutions Evaluation Matrix	14
Table 3.6	Design Trade-off Analysis	16
Table 3.7	Sustainability, Ethics, Safety and Security Considerations	18
Table 4.1	Risk Register	19
Table 4.2	Software Tools and Resources Used	20
Table 4.3	Test Plan and Verification Criteria	22
Table 4.4	Test Cases and Results	24
Table 4.5	Specification and Achieved Values	25
Table 4.6	Project Team Roles and Contributions	26
Table 4.7	Estimated and Actual Project Cost	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
GA	Gate Assignment	-
GAAS	Airport Gate Assignment System	-
AT	Arrival Time	hh:mm
DT	Departure Time	hh:mm
GT	Gate	-
ERT	Earliest Release Time	hh:mm
FCFS	First-Come, First-Served	-
ALG	Algorithm	-
CPU	Central Processing Unit	-
RAM	Random Access Memory	-
GREEDY	Greedy Selection	-

CHAPTER 1

INTRODUCTION AND ANALYTICAL PROBLEM

1.1 Background

Airports handle a large number of flights within limited terminal space and therefore require efficient allocation of gates. Each flight occupies a gate from its arrival time until its departure time, and two overlapping flights cannot normally use the same gate at the same time. Gate assignment can therefore be treated as a scheduling and resource-allocation problem.

The proposed project develops an Airport Gate Assignment System using a greedy algorithm. Flight records are organized according to arrival time, and each flight is assigned to a gate that becomes available earliest while satisfying the flight's schedule. The approach demonstrates how a locally optimal decision can be used to construct a practical schedule.

1.2 Need for the Project

Airport gate scheduling becomes difficult when many flights arrive and depart within overlapping time intervals. Manual allocation may lead to gate conflicts, unused resources, and avoidable delays. A structured algorithmic approach is therefore required to assign gates consistently and efficiently.

The project is needed to:

- Understand basic Greedy Algorithm concepts practically.
- Demonstrate flight-data validation and gate assignment step by step.
- Observe gate and flight schedule storage changes.
- Reduce dependency on physical scheduling system hardware.
- Provide an easy educational tool for students to understand scheduling and resource allocation.

1.3 Problem Statement

The problem is to develop a reliable airport gate assignment system that allocates flights to available gates without schedule conflicts. The system should accept flight arrival and departure times, process flights in a suitable order, select an available gate using a greedy strategy, handle cases where all gates are occupied, and present the final assignment clearly.

1.4 Project Aim

The main aim of this project is to design and implement an airport gate assignment system using a greedy algorithm, with emphasis on conflict-free scheduling, efficient gate utilization, clear decision-making, and practical evaluation of the resulting assignments.

1.5 Project Objectives

1. To design a simplified scheduling system model.
2. To develop a basic flight-data validation and gate assignment mechanism.
3. To model the fetch, decode, and execute cycle.
4. To implement arithmetic, logical, and data-transfer flights.
5. To display changes in flight records and flight schedule records.
6. To test the system using different flight sequences.
7. To evaluate the correctness of flight-data validation and gate assignment.

1.6 Scope

Included

- Basic scheduling system simulation.
- Gate and flight schedule storage representation.
- Program Counter management.
- Flight fetch and execution.
- Arithmetic and logical operations.
- Data transfer flights.
- Basic branching flights.
- Display of execution results.

Excluded

- Complete commercial scheduling system simulation.
- Physical hardware implementation.
- Transistor-level scheduling system simulation.
- Advanced pipeline and cache simulation.

Assumptions

- A predefined feature set is used.
- Gates and flight schedule storage have fixed sizes.
- Flights follow a predefined syntax.

Boundary Conditions

The system operates only with the flights, gates, and scheduling constraints defined in the project.

1.7 Expected Engineering Outcome

The expected outcome is a functional scheduling prototype that accepts flight schedules, applies a greedy gate-assignment strategy, produces conflict-free allocations when sufficient gates are available, and provides clear evidence of gate utilization and scheduling performance.

The system is expected to:

- Execute supported flights.
- Update flight records and flight schedule records.
- Maintain the program counter.
- Demonstrate the fetch-decode-execute cycle.
- Display scheduling system-state changes.
- Identify invalid or unsupported flights.

The final outcome will be a software prototype and educational tool for understanding greedy algorithms, interval scheduling, and airport resource allocation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SCHEDULING SOLUTIONS

2.1 Review Method

Relevant information was reviewed from algorithm-design literature, scheduling studies, airport operations concepts, and documentation related to greedy algorithms. The review focused on interval scheduling, resource allocation, sorting, gate availability, conflict detection, computational complexity, and practical scheduling constraints. Existing approaches were compared based on simplicity, computational efficiency, scalability, and suitability for airport gate allocation.

2.2 Review of Existing Work

Existing scheduling approaches commonly use sorting, priority queues, interval scheduling, and optimization techniques to allocate limited resources. Greedy methods are widely used when a sequence of locally optimal choices can produce an efficient feasible solution. For gate assignment, maintaining the earliest gate-release time provides a practical way to decide which gate should receive the next flight.

Traditional airport scheduling systems may include real-time operational constraints, passenger connections, aircraft towing, terminal restrictions, and disruption management. Such systems can be complex for beginners because they combine many operational variables.

Educational scheduling systems provide simplified flight and resource models. These systems are easier to understand and are suitable for demonstrating fundamental algorithmic concepts such as sorting, greedy selection, and resource tracking.

Exact optimization tools can produce highly optimized schedules, but their main purpose is solving large operational problems rather than clearly explaining the greedy decision process to beginners.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual Gate Allocation	Simple for small schedules	Prone to conflicts and human error	Baseline for comparison
Optimization-Based Scheduling	Can handle many constraints	More complex and computationally demanding	Reference for advanced extension

Priority-Queue Scheduling	Efficient gate-status tracking	Requires additional data structures	Useful for implementation improvement
Proposed Greedy Algorithm	Simple, fast and transparent	Limited for complex operational constraints	Directly addresses the project objective

2.4 Research / Engineering Gap

Existing airport scheduling systems can provide detailed operational planning, but their complexity can make the underlying allocation logic difficult for beginners to understand. Simple examples, on the other hand, may not demonstrate gate conflicts and resource availability clearly.

The engineering gap is the need for a clear and reproducible educational workflow that demonstrates how a greedy algorithm can allocate limited airport gates while tracking flight intervals and conflicts. The proposed project addresses this by integrating flight sorting, gate availability checking, greedy selection, assignment updates, and performance analysis in one workflow.

2.5 Proposed Contribution

The proposed system provides a structured scheduling workflow for airport gate allocation. It combines flight sorting, interval comparison, greedy gate selection, and result evaluation so that users can observe how each scheduling decision affects gate availability.

The main contribution is an educational scheduling environment that demonstrates the relationship between flight intervals, gate availability, and greedy decisions. The system can be extended in the future to support gate priorities, terminal constraints, aircraft categories, delays, and advanced optimization methods.

CHAPTER 3

ALGORITHM DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Students	Easy-to-understand flight and gate assignment
Faculty	Demonstration of greedy algorithm concepts
Project Team	Simple and reliable gate assignment system
Users	Clear gate assignments and availability information

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Flight processing	Functional	Correct processing order
Gate assignment	Functional	Conflict-free allocation
Gate status operation	Functional	Update correctly
Error handling	Functional	Detect invalid schedule data
Usability	Non Functional	Clear scheduling output

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 29148	Requirements specification	Used to define system requirements
ISO/IEC 25010	Software quality	Used for usability, reliability, and maintainability

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Number of gates	Configurable gate set	Number	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Exact Optimization-Based Gate Assignment System

An optimization-based airport scheduling system can consider many constraints and search for globally optimized assignments. However, it is more complex to implement and difficult for beginners to trace step by step.

Alternative 2: Greedy Airport Gate Assignment System

A simplified greedy gate assignment system processes flights in arrival order and assigns an earliest available gate. It is easy to develop, understand, and suitable for demonstrating scheduling concepts.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	4	5	4
Cost	20%	2	5	4

3.5 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Greedy Algorithm	Optimization / exhaustive search	Simple and efficient scheduling	May not guarantee global optimum with complex constraints	Selected
Software	Manual gate allocation	Low cost Low cost and repeatable	Requires valid input data	Selected
Earliest-available gate	First available gate without ordering	Good gate utilization	Needs gate-status tracking	Selected

3.6 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Efficient software-based scheduling	Scheduling system	Reduced manual coordination and resource waste
Ethics	Algorithmic and academic integrity	Proper references and test-data	Responsible interpretation

		acknowledgement	
Health & Safety	Software and scheduling based	No physical intervention	Low physical risk
Security	Input validation	Time validation and conflict checking	Safe handling of schedule data

Risk Gate

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Missing or invalid flight time	medium	medium	Medium	Input validation	Low
Incorrect flight sorting	medium	High	Medium	Sorting and test-case verification	Low
Gate availability conflict	low	High	Low	Availability checks and boundary validation	Low

CHAPTER 4

IMPLEMENTATION, SYSTEM TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative algorithm-design approach. First, the flight and gate requirements were defined. Next, flight records were validated and sorted by arrival time. The greedy assignment procedure was then implemented to select an available gate and update its release time. Testing was performed using different flight schedules and gate counts.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The system was implemented using Python. The workflow accepts flight and gate data, sorts flights by arrival time, checks gate release times, applies greedy gate selection, updates assignments, and displays the final schedule. Standard Python data structures such as lists and sorting operations are sufficient for the prototype.

Tool / Software / Equipment	Purpose	Stage Used
Python	Program	Implementation
Python IDLE / Python environment	Python code	Testing and evaluation
Computer	Development, analysis and testing	All stages

[Insert airport gate assignment prototype screenshot / final implementation evidence here.]

4.3 Test Plan & Verification

Requirement	Test Method		Acceptance Criterion
Flight input validation	Check arrival/departure times		Valid schedule data
Gate assignment	Check assigned gate		Conflict-free assignment
System specification	Required gate allocation	Achieved gate allocation	Status (Pass/Fail)
Flight sorting and gate assignment	Correct	Correct	Pass
Gate assignment	Conflict-free	Conflict-free	Pass

4.4 Results

The developed system successfully processed the selected flight schedules and assigned flights to available gates using the greedy algorithm. Testing showed that flights were assigned according to arrival order and that a gate was reused only after its previous flight had departed.

Insert here:

- System output screenshot
- Test result table
- Execution output

[Insert gate assignment results tables / graphs / scheduling outputs here.]

4.5 Data Analysis & Engineering Judgment

The test results show that the greedy workflow produces valid gate assignments when sufficient gates are available. Comparing arrival times with gate release times provides evidence that overlapping flights are not assigned to the same gate. The approach also demonstrates efficient decision-making with simple and transparent scheduling logic.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design airport gate assignment workflow	Flight and gate model	Fully Achieved
Flight sorting and greedy assignment workflow	Step-by-step scheduling processing	Fully Achieved
Test the flight schedules and gate assignments	Test schedules and gate cases	Fully Achieved
Evaluate scheduling results	Conflict checks and gate utilization	Fully Achieved

4.6 Strengths & Limitations

Strengths:

- Simple and easy to use.
- Low-cost software implementation.
- Step-by-step flight-data validation and gate assignment.
- Useful for learning Greedy Algorithm concepts.

Limitations:

- Supports only a limited feature set.
- Does not simulate physical hardware.
- Advanced pipeline and cache flight records are not included.

Project Planning, Roles & Budget

[Insert Gantt chart / milestone timeline here.]

Student	Assigned Responsibility	Actual Contribution
K Harshika	System design, implementation, testing & report preparation	Designed the gate-assignment architecture, implemented and tested the greedy scheduling algorithm, and prepared the project documentation
Item	Estimated Cost	Actual Cost
Software	₹0 ₹0	₹0 ₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Airport Gate Assignment System was successfully designed and implemented to demonstrate the use of a greedy algorithm for flight-to-gate scheduling. The system processes flight arrival and departure times, checks gate availability, assigns each flight to a suitable gate, and updates gate release times. Testing confirmed that the implemented scheduling logic produced valid assignments for the selected test cases. The project provides a simple and useful learning tool for understanding greedy algorithms, scheduling, and resource allocation.

5.2 Future Work

- Add more advanced flights.
- Support different Greedy Algorithm architectures.
- Add a graphical user interface.
- Include pipeline and cache simulation.
- Provide optimization-based comparison and detailed scheduling visualizations.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Understanding of Greedy Algorithm and scheduling system architecture.
- Knowledge of flight-data validation and gate assignment.
- Understanding of flight records and flight schedule records.
- Practical Python programming experience.

Engineering & Professional Skills Developed

- System design and implementation.
- Programming and debugging.
- Testing and problem solving.

- Teamwork and technical documentation.

Individual Reflection – K Harshika

- The main difficulty was understanding flight-data validation and gate assignment and scheduling system states.
- I contributed to the system design and planning.
- I learned how flight records and flights interact during execution.
- If repeated, I would include more flights and better visualization.
- The main challenge was implementing the flight-data validation and gate assignment logic.
- I contributed to coding and testing the system.
- I improved my Python programming and debugging skills.
- If repeated, I would add a graphical interface.
- The main challenge was testing different flight sequences.
- I contributed to testing and report preparation.
- I learned about Greedy Algorithm, flight records, flight schedule storage, and scheduling system operations.
- If repeated, I would improve the system with larger schedules and additional airport constraints.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.
- [2] S. S. Skiena, The Algorithm Design Manual, Springer.
- [3] E. Horowitz, S. Sahni and S. Rajasekaran, Computer Algorithms, Computer Science Press.
- [4] Python Software Foundation, Python Documentation, Python Software Foundation.
- [5] IEEE, IEEE Standard for Software and Systems Engineering—Life Cycle Processes—Requirements Engineering, IEEE Standard 29148.
- [6] ISO/IEC, ISO/IEC 25010: Systems and Software Quality Requirements and Evaluation (SQuaRE), International Organization for Standardization.
- [7] T. H. Cormen et al., selected algorithm analysis concepts for scheduling and greedy methods.
- [8] GitHub, GitHub Documentation, GitHub, Inc.

APPENDICES

Appendix A – Detailed Calculations

Sample gate-availability calculations and scheduling results used for verifying the greedy workflow.

Example:

Flight A: 09:00–10:00; Flight B: 10:15–11:00 → the same gate can be reused because $10:00 \leq 10:15$.

Appendix B – Engineering Drawings / Schematics

Include the airport gate assignment workflow diagram.

Flight Input → Validate → Sort by Arrival → Check Gates → Greedy Selection → Update Release Time → Final Assignment

Appendix C – Source Code / Code Repository Information

The complete Python source code for the airport gate assignment system is maintained separately. Essential sections for flight input, sorting, gate availability checking, greedy selection, assignment updates, and result display may be included here.

Appendix D – Datasheets

Not applicable, since the project is a software scheduling study and does not require physical hardware.

Appendix E – Experimental / Raw Data

Include flight schedules, gate counts, gate-release calculations, test cases, conflict checks, and final assignment outputs used during verification.

Appendix F – Ethics / Institutional Approval

The project should use appropriately documented test schedules and follow applicable data-privacy, safety, and academic-integrity requirements. Operational airport data, if used in future work, should be protected and handled responsibly.

Appendix G – Project Meeting / Progress Records

Include project discussion records, milestone updates, and progress screenshots, where applicable.

Appendix H – User/Industry Feedback

Not applicable, unless feedback was collected from users or faculty.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable, if the project has no publication, patent, competition, or product outcome.

Appendix J – Individual Contribution Evidence

Student	Contribution Evidence
---------	-----------------------

K Harshika	System design, coding, implementation, testing, and report documentation
------------	--

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)